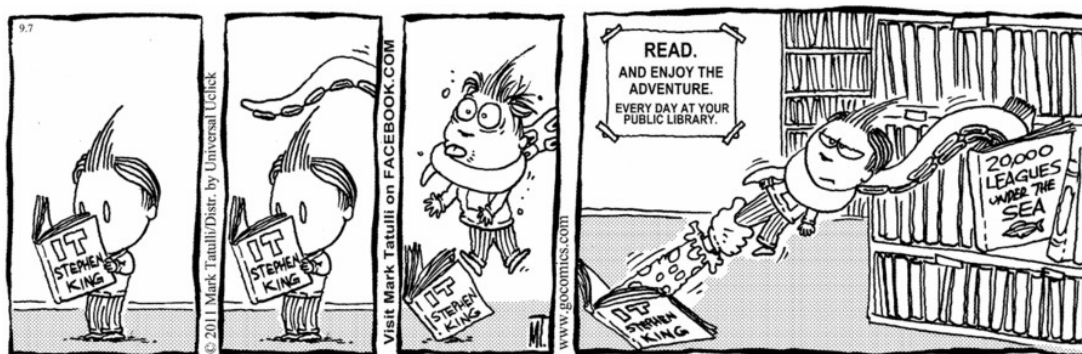


Cahier des charges

Réalisation d'un programme en C

« La bibliothèque »



Superviseurs :

Cauchie A.
(a.cauchie@indbg.be)

Embrechts L.
(l.embrechts@indbg.be)

Réalisé par :
Strale Robin

1. Définition du projet

1.1. Description de la problématique

Une bibliothèque communautaire fait appel à vous afin de créer un système informatique pour gérer son stock et la location de ses livres. L'objectif de ce projet est de mettre en place un logiciel qui réponde le plus possible aux attentes des commanditaires.

Une rencontre avec le président et les différents bénévoles a permis de déterminer ce qu'ils souhaiteraient mettre en place. Cette rencontre a été retranscrite par écrit et est disponibles dans les annexes de ce document.

1.2. Délivrables

1.2.1. Échéances

Délivrable	Échéance
Interfaces et cas d'utilisation	19 mars
Description des données	26 mars
Diagramme(s) d'action	2 avril
La structure générale du code (prototypes des fonctions, structures, etc.)	16 avril
L'implémentation des fonctions	14 mai
Le projet complet	19 mai

1.2.2. Consignes de remise

Le programme (code source, compilé et exécutable) et toutes les ressources nécessaires à son exécution ainsi que le cahier des charges (format PDF) doivent être remis dans un dossier nommé LangC_5TT_Nom_Prénom. Ce dossier sera déposé sur la plateforme « Classroom ». **Aucun retard ne sera accepté sauf cas de force majeure.** En cas de non remise à la date indiquée, une cote nulle sera attribuée.

1.2.3. Évaluation

Évaluation du cahier des charges

Le cahier des charges (ce document) doit être complété. La partie 1 est complète, à vous de compléter les autres.

La mise en page, l'orthographe et la qualité de la rédaction seront prises en compte dans l'évaluation.

Évaluation du programme

Le programme sera évalué en fonction des points suivants :

- le programme s'exécute correctement au niveau des fonctionnalités attendues,

- le code source est de qualité ou non

Des bonus seront attribués pour :

- l'allocation dynamique
- ou des fonctionnalités supplémentaires non mentionnées dans le cahier des charges

Toutefois, ces bonus ne seront attribués que si la note du travail de base atteint les 80%.

Évaluation orale

L'évaluation orale consiste à présenter le projet oralement devant les superviseurs. La présentation se déroulera comme il suit :

1. Présentation du logiciel
2. Présentation du code source
3. Crash test : un autre élève teste (sans consigne du développeur) le logiciel, sa robustesse, sa prise en main
4. Questions / Réponses

La présentation du logiciel ainsi que la présentation du code source ne devra pas dépasser 10 minutes.

1.3. Technologies utilisées

1.3.1. Langage de programmation

Le programme doit être rédigé en langage C.

1.3.2. Bibliothèques

Aucune bibliothèque supplémentaire non vue a cours ne peut être utilisée sauf accord des superviseurs.

2. Analyse et conception

2.1. Interfaces et cas d'utilisation

Interfaces principale :

- I. Gestion du stock
 - II. Gestion des emprunts
 - III. Gestion des membres
 - IV. Statistiques
 - V. Quitter
-

Entrer un numéro de 1 à 4 :

Interfaces Gestion du stock :

- I. Ajouter un livre au stock
 - II. Supprimer un livre du stock
 - III. Modifier un livre du stock
 - IV. Lister les ouvrages du stock
 - V. Rechercher un livre dans le stock
 - VI. Quitter
-

Entrer un numéro de 1 à 6 :

Interface Ajouter un livre au stock :

Nom du livre à ajouter : **Guide Michelin 2023 France**

Auteur du livre : **Michelin**

Éditeur du livre : **Michelin Travel Partner**

Année de parution : **2023**

(message d'erreur : « Date invalide »)

Code ISBN (10 chiffres) : **2067257412**

(message d'erreur : « Code ISBN invalide »)

(message d'erreur si il existe déjà :

« Ce livre est déjà en stock »)

Voulez-vous ajouter ce livre ? : (o/n)

Le livre a bien été ajouté

Interface Supprimer un livre du stock :

Entrer le code ISBN a supprimer : **2067257412**

(message d'erreur : « Code ISBN invalide »)

Voulez-vous supprimer ce livre ? : o/n

Le livre a bien été supprimé

Interface Modifier un livre du stock :

Quel livre souhaitez-vous recherche ? Entrer l'ISBN: 2067257412

(message d'erreur si l'ISBN est invalide)

I.Modifier l'ISBN

Entrer le code ISBN actuel: 2067257412

Entrer le nouveau code : 2067257413

À l'intérieur de Modifier l'ISBN

(message d'erreur si l'ISBN existe déjà)

Voulez-vous modifier ce code : o/n

L'ISBN a bien été modifié

II. Modifier le titre

Entrer le titre du livre actuel: Guide Michelin 2023 France

Entrer le nouveau titre : Guide Michelin 2023 France (Poche)

À l'intérieur de Modifier le titre

(message d'erreur si le titre existe déjà)

Voulez-vous modifier ce titre ? : o/n

Le livre a bien été modifié

III.Modifier l'auteur

Entrer l'auteur actuel: Michelin

Entrer le nouvel auteur : Micheline

À l'intérieur de Modifier l'auteur

Voulez-vous modifier l'auteur du livre ? : o/n

L'auteur a bien été modifié

IV.Modifier l'année de parution

Entrer l'année de parution actuelle: 2023

Entrer la nouvelle année de parution : 2022

À l'intérieur de Modifier l'année De parution

(message d'erreur si l'année de parution est invalide)

Voulez-vous modifier cette année ? : o/n

L'année a bien été modifiée

V. Modifier l'éditeur

Entrer l'éditeur actuel: Michelin Travel Partner

Entrer le nouvel éditeur : Michelin Travel Partners

À l'intérieur de Modifier l'éditeur

Voulez-vous modifier cet editeur ? : o/n

L'éditeur a bien été modifié

Entrer votre choix :

Interface Lister les livres du stock :

Libre| 2266111665| Memoires d'Hadrien| Marguerite Yourcenar|1951| Gallimard

Interface Rechercher un livre dans le stock :

Quel livre souhaitez-vous recherche ? Entrer l'ISBN: 2067257412

Libre| 2067257412|Guide Michelin 2023 France (Poche)| Micheline|2022| Michelin Travel Partners

Interfaces Gestion des emprunts :

I. Emprunter des livres

II. Remettre des livres

III. Payer une amende

IV. Rechercher un livre

V. Afficher la liste des livres empruntés

VI. Retour au menu

Entrer un nombre de 1 à 6:

Interface Emprunter des livres :

Entrer le numéro du membre : 12

(message d'erreur si le membre est introuvable ou si il a emprunté trop de livres)

Entrer l'ISBN du livre à emprunter : 2067257412

Voulez-vous emprunter ce livre ? : (o/n)

Le livre a bien été emprunté

Le prix s'élève à : 0,75€

Interface Remettre des livres :

Entrer le numéro du membre qui veut rendre le livre: 12

Entrer le code ISBN du livre a rendre : 2067257412

(message d'erreur si l'emprunt est introuvable)

Voulez-vous rendre ce livre ? o/n

Le livre a bien été rendu

Interface Payer une amende :

Entrez le numéro du membre qui veut payer une amende: 12

Entrez l'ISBN du livre : 2067257412

(message d'erreur si l'ISBN est introuvable)

Amende à payer : 0,10€

Voulez-vous payer l'amende ? o/n

L'amende a bien été payée

Interface Rechercher un livre :

Quel emprunt souhaitez - vous recherche ? Entrer l'ISBN: 2067257412

(message d'erreur si l'ISBN est introuvable)

20230621| 12| 2266111568

Interface Afficher la liste des emprunts :

20230621| 12| 2266111568

20230621| 12| 2266111567

20230621| 12| 2266111569

Interface Gestion des membres :

- I. Identifier un membre
 - II. Ajouter un membre
 - III. Supprimer un membre
 - IV. Modifier les données d'un membre
 - V. Afficher la liste des membres
 - VI. Retour au menu
-

Entrer un nombre de 1 à 6:

Interface Identifier un membre :

Entrer le numéro du membre : 12

(message d'erreur si l'ISBN est introuvable)

Strale| Robin| Rue de dinant, 34 Falmignoul| 20230621

Interface Ajouter un membre :

Entrez le numéro du membre : 12

(message d'erreur si le numéro du membre existe déjà)

Entrer le nom du membre : Strale

Entrer le prénom du membre : Robin

Entrer l'adresse du membre : Rue de dinant, 34 Falmignoul

Voulez-vous ajouter ce membre ? : (o/n)

Le membre a bien été ajouté

Interface Supprimer un membre :

Entrez le numéro du membre à supprimer : 12

(message d'erreur si le membre n'existe pas)

Voulez-vous supprimer ce membre ? : o/n

Le membre a bien été supprimé

Interface Modifier les données d'un membre :

Entrez le numéro du membre a supprimer : 12

(message d'erreur si le membre n'existe pas)

I.Modifier le numéro du membre

Entrer le numéro actuel du membre: 12

Entrer le nouveau numero : 13

À l'intérieur de Modifier le Numéro du membre

(message d'erreur si le numéro existe déjà)

Voulez-vous modifier ce numéro ? : o/n

Le numéro a bien été modifié

II. Modifier le nom du membre

Entrer le nom actuel du membre: Strale

À l'intérieur de Modifier le Nom du membre

Entrer le nouveau nom : Ilegems

Voulez-vous modifier ce nom ? : o/n

Le nom a bien été modifié

III.Modifier le prénom du membre

Entrer le prénom actuel du membre: Robin

À l'intérieur de Modifier le prénom du membre

Entrer le nouveau prénom : Ferre

Voulez-vous modifier ce prénom ? : o/n

Le prénom a bien été modifié

IV.Modifier l'adresse du membre

Entrer l'adresse actuelle du membre: Rue dinant, 34 Falmignoul

À l'intérieur de Modifier l' adresse du membre

Entrer la nouvelle adresse : Rue des chrétias

Voulez-vous modifier cette adresse ? : o/n

L'adresse a bien été modifiée

Entrer votre choix :

Interface Afficher la liste des membres:

13| Ilegems| Ferre| Rue des chrétias

17| Godart| Pavel| Malvoisin

2.2. Description des données

Fichier pour les ouvrages :

Le nom du fichier est : `ouvrages.dat`

Le fichier contient des livres qui sont classés par leur `code ISBN`.

Son type est un `fichier.dat`

Propriété	Type	Description
Code	Chaîne de caractère [11]	Chaîne de caractère qui contient le code ISBN du livre
Titre	Chaîne de caractère [100]	Chaîne de caractère qui contient le titre de l'ouvrage
Auteur	Chaîne de caractère [100]	Chaîne de caractère qui contient le nom de l'auteur
Date	INT	Entier qui référence l'année de parution
Éditeur	Chaîne de caractère [50]	Chaîne de caractère qui contient le nom de la maison d'édition

Fichier pour les membres :

Le nom du fichier est : `membres.dat`

Ce fichier contient toutes les données concernant la gestion des membres.

Le type du fichier : `fichier.dat`

Propriété	Type	Description
Nom et Prénom du membre	Chaîne de caractère [100]	Chaîne de caractère qui contient le nom et le prénom du membre
Numéro du membre	INT	Entier qui contient le numéro du membre
Adresse	Chaîne de caractère [100]	Chaîne de caractère qui contient l'adresse du membre
Date d'adhésion	INT	Entier qui contient la date d'adhésion du membre en (AAAA-MM-JJ)

Fichier pour les emprunts :

Le nom du fichier : **emprunts.dat**

Ce fichier contient toutes les données concernant la gestion des emprunts.

Le type du fichier est : **fichier.dat**

Propriété	Type	Description
Date d'emprunt	INT	Entier qui contient la date de parution en (AAAA-MM-JJ)
Numéro du membre	INT	Entier qui contient le numéro du membre
Code	Chaîne de caractère [11]	Chaîne de caractère qui contient le code ISBN du livre

2.3. Diagramme(s) d'action

Diagramme d'action de la fonction main() ;

module(FctOuvertureFichiers;pFichierEmprunts,pFichierMembres,pFichierLivres;pFichierEmprunts,pFichierMembres,pFichierLivres)

if (pFichierLivres != nil)

module(DemanderEtAfficherChoix;; numero_Choix)

while (numero_Choix != 0)

sortir "1. Gestion du stock"

```
sortir "2. Gestion des emprunts"
sortir "3. Gestion des membres"
sortir "0. Quitter"
sortir "-----"

if (numero_Choix = 1 )
    module(GestionDesLivres)

elseif (numero_Choix = 2)
    module(GestionDesEmprunts)
elseif (numero_Choix = 3)
    module(GestionDesMembres)
else
    sortir "Le chiffre n'existe pas. Veuillez entrer un nombre entre 0 et 3"
endif
module(DemanderEtAfficherChoix;; numero_Choix)
endwhile
module(FctFermetureFichiers)
```

3. Implémentation

```
#include "header.h"
```

```
void main(void) {  
    int numero_choix;  
    if (FctOuvertureFichiers()) {  
        numero_choix = AfficheMenuEtDemandeChoix();  
  
        while (numero_choix >= 1 && numero_choix < 4)  
        {  
  
            switch (numero_choix) {  
                case 1: GestionDuStock();  
                        break;  
                case 2: GestionDesEmprunts();  
                        break;  
                case 3: GestionDesMembres();  
                        break;  
            }  
            numero_choix = AfficheMenuEtDemandeChoix();  
        }  
    }  
}
```

```
// Fonction qui ouvre les fichiers
```

```
bool FctOuvertureFichiers(void) {
```

```

// Fichier Emprunts

FILE* pTabEmprunts;

fopen_s(&pTabEmprunts, "emprunts.dat", "rb+");

if (pTabEmprunts == NULL) {

    fopen_s(&pTabEmprunts, "emprunts.dat", "ab+");

    fclose(pTabEmprunts);

    fopen_s(&pTabEmprunts, "emprunts.dat", "rb+");

}

fclose(pTabEmprunts);


// Fichiers Membres

FILE* pTabMembres;

fopen_s(&pTabMembres, "membres.dat", "rb+");

if (pTabMembres == NULL) {

    fopen_s(&pTabMembres, "membres.dat", "ab+");

    fclose(pTabMembres);

    fopen_s(&pTabMembres, "membres.dat", "rb+");

}

fclose(pTabMembres);


// Fichiers Livres

FILE* pTabLivres;

fopen_s(&pTabLivres, "ouvrages.dat", "rb+");

if (pTabLivres == NULL) {

    printf("Fichier Introuvable");

    system("pause");

    return false;

}

```

```

        fclose(pTabLivres);

        return true;

};

// Fonctions principales
void GestionDuStock(void) {

    int choixStock = AfficherEtDemandeChoixStock();
    while (choixStock >= 1 && choixStock < 6)
    {

        switch (choixStock) {
            case 1: ajouterLivreAuStock();
                    break;
            case 2: SupprimerLivreDuStock();
                    break;
            case 3: ModifierLivreStock();
                    break;
            case 4: AfficherLesLivresDuStock();
                    break;
            case 5: RechercherLivreDansStock();
                    break;
            case 6: choixStock = AfficheMenuEtDemandeChoix();
        }

        choixStock = AfficherEtDemandeChoixStock();

    }

}

```

```

void GestiondesEmprunts(void) {
    int choixLivre = AfficherEtDemandeChoixEmprunts();

    while (choixLivre >= 1 && choixLivre < 6)
    {

        switch (choixLivre) {
            case 1: EmprunterLivre();
                    break;
            case 2: RemiseDesLivres();
                    break;
            case 3: PayerAmende();
                    break;
            case 4: RechercherUnLivre();
                    break;
            case 5: AfficherListeLivresEmpruntes();
                    break;
            case 6: choixLivre = AfficheMenuEtDemandeChoix();

        }

        choixLivre = AfficherEtDemandeChoixEmprunts();

    }
}

void GestionDesMembres(void) {
    int choixMembre = AfficherEtDemandeChoixMembres();

    while (choixMembre >= 1 && choixMembre < 6)
    {

```



```

        switch (choixMembre) {
        case 1: IdentifierMembre();
                break;
        case 2: AjouterMembre();
                break;
        case 3: SupprimerMembre();
                break;
        case 4: ModifierMembre();
                break;
        case 5: AfficherListeMembres();
                break;
        case 6: choixMembre = AfficheMenuEtDemandeChoix();
        }

        choixMembre = AfficherEtDemandeChoixMembres();

    }
}

```

// Fonction qui demande et qui affiche les menus

```

int AfficheMenuEtDemandeChoix(void) {
    int numero_choix;

    printf("I. Gestion du stock\n");
    printf("II. Gestion des emprunts\n");
    printf("III. Gestion des membres\n");
    printf("IV. Quitter\n");
    printf("-----\n");

    printf("Entrer un nombre de 1 a 4:");
    scanf_s("%d", &numero_choix);
}

```

```

        system("cls");

        return numero_choix;

    }

    int AfficherEtDemandeChoixStock(void) {
        int choixStock;

        printf("I. Ajouter un livre au stock\n");
        printf("II. Supprimer un livre du stock\n");
        printf("III. Modifier un livre du stock\n");
        printf("IV. Lister les ouvrages du stock\n");
        printf("V. Rechercher un livre dans le stock\n");
        printf("VI. Retour au menu\n");
        printf("-----\n");

        printf("Entrer un nombre de 1 a 6:");
        scanf_s("%d", &choixStock);
        viderTampon();
        system("cls");

        return choixStock;
    }

    int AfficherEtDemandeChoixEmprunts(void) {
        int choixLivre;

        printf("I. Emprunter des livres\n");
        printf("II. Remettre des livres\n");
        printf("III. Payer une amende\n");
        printf("IV. Rechercher un livre\n");
        printf("V. Afficher la liste des livres empruntes\n");
        printf("VI. Retour au menu\n");
    }

```

```

printf("-----\n");

printf("Entrer un nombre de 1 a 6:");
scanf_s("%d", &choixLivre);
system("cls");

return choixLivre;
}

int AfficherEtDemandeChoixMembres(void) {
    int choixMembre;

    printf("I. Identifier un membre\n");
    printf("II. Ajouter un membre\n");
    printf("III. Supprimer un membre\n");
    printf("IV. Modifier les donnees d'un membre\n");
    printf("V. Afficher la liste des membres\n");
    printf("VI. Retour au menu\n");
    printf("-----\n");

    printf("Entrer un nombre de 1 a 6:");
    scanf_s("%d", &choixMembre);
    system("cls");

    return choixMembre;
}

// fonctions pour la gestion du stock
void ajouterLivreAuStock(void) {
    FILE* pTabLivres;
    Livre livreAjout;
    Livre livreBD;

```

```

char validation;

int dateAjd = obtenir_annee();

bool nouvelISBN;

fopen_s(&pTabLivres, "ouvrages.dat", "ab+");

printf("Nom du livre a ajouter:\n");
red();
gets_s(livreAjout.titre, TAILLE_TITRE);
reset();

printf("Auteur du livre :\n");
red();
gets_s(livreAjout.auteur, TAILLE_AUTEUR);
reset();

printf("Editeur du livre : \n");
red();
gets_s(livreAjout.editeur, TAILLE_EDITEUR);
reset();

printf("Annee de parution:\n");
red();
scanf_s("%d", &livreAjout.date);
viderTampon();
reset();

while (livreAjout.date > dateAjd) {
    printf("Année incorrect\n");
    red();
}

```

```

scanf_s("%d", &livreAjout.date);
reset();
viderTampon();
}

```

```

fclose(pTabLivres);

```

```

do

```

```

{
    nouvelISBN = true;
    fopen_s(&pTabLivres, "ouvrages.dat", "ab+");

```

```

    printf("Entrez un ISBN :\n");
    red();
    scanf_s("%s", livreAjout.code, TAILLE_CODE);
    reset();
    viderTampon();

```

```

    while (strlen(livreAjout.code) != 10)

```

```

    {
        printf("ISBN invalide\n");
        printf("Entrez le code ISBN du livre :\n");
        red();
        scanf_s("%s", livreAjout.code, TAILLE_CODE);
        reset();
        viderTampon();
    }

```

```

    while (fread_s(&livreBD, sizeof(livreBD), sizeof(livreBD), 1, pTabLivres) !

```

```

    = 0)

```

```

    {

```

```

        if (strcmp(livreAjout.code, livreBD.code) == 0)
        {
            printf("L'ISBN existe deja\n");
            nouvelISBN = false;
        }
    }
    fclose(pTabLivres);

} while (!nouvelISBN);

fopen_s(&pTabLivres, "ouvrages.dat", "ab+");

printf("Voulez-vous ajouter ce livre ? : (o/n)\n");
green();
scanf_s("%c", &validation, 1);
reset();
viderTampon();

if (validation == 'o') {
    printf("Ce/Ces livres ont bien ete ajoute(s)\n");
    fwrite(&livreAjout, sizeof(livreAjout), 1, pTabLivres);
}
else {
    printf("Ce livre n'a pas ete ajoute\n");
    system("cls");
}

fclose(pTabLivres);

};

```

```

void SupprimerLivreDuStock(void) {
    FILE* pTabLivres;
    Livre livreSupprime;
    Livre livre;
    char validationSupprime;
    int numero_choix;

    fopen_s(&pTabLivres, "ouvrages.dat", "rb+");

    printf("Entrer le code ISBN a supprimer :\n");
    red();
    gets_s(livreSupprime.code, TAILLE_CODE);
    reset();

    while (fread_s(&livre, sizeof(livre), sizeof(livre), 1, pTabLivres) != 0 &&
    strcmp(livre.code, livreSupprime.code) != 0) {

        };

    if (feof(pTabLivres)) {
        printf("ISBN introuvable\n");

    }
    else {
        printf("Voulez-vous supprimer ce livre ?: o/n\n");
        green();
        scanf_s("%c", &validationSupprime, 1);
        viderTampon();
        reset();
    }
}

```

```

        if (validationSupprime == 'o') {
            printf("Le livre a bien ete supprime\n");
        }

        else {
            system("cls");

        }

        strcpy_s(livre.code, TAILLE_CODE, "****");
        fseek(pTabLivres, -1 * (long)sizeof(livre), SEEK_CUR);
        fwrite(&livre, sizeof(livre), 1, pTabLivres);

    }

    fclose(pTabLivres);

};

void ModifierLivreStock(void) {
    FILE* pTabLivres;
    Livre livreModifie;
    Livre newLivre;
    char validerModif;
    int choixStock;
    int choixModification;

    RechercherLivreDansStock();
    fopen_s(&pTabLivres, "ouvrages.dat", "rb+");

    printf("I.Modifier l'ISBN\n ");
    printf("II. Modifier le titre\n");

```



```
printf("III.Modifier l'auteur\n");
printf("IV.Modifier l'annee de parution\n");
printf("V. Modifier l'editeur\n");
```

```
printf("Entrer votre choix : \n");
red();
scanf_s("%d", &choixModification);
viderTampon();
reset();
system("cls");
```

```
switch (choixModification) {
case 1:
```

```
    printf("Entrer le code Isbn actuel: \n");
    red();
    gets_s(livreModifie.code, TAILLE_CODE);
    reset();
```

```
        while (fread_s(&newLivre, sizeof(newLivre), sizeof(newLivre), 1,
pTabLivres) != 0 && strcmp(newLivre.code, livreModifie.code) != 0) {}
```

```
        if (feof(pTabLivres)) {
            printf("ISBN introuvable");
```

```
        }
        else {
```

```
            printf("Entrer le nouveau code :\n");
            red();
            gets_s(newLivre.code, TAILLE_CODE);
            reset();
```

```
            printf("Voulez-vous modifier ce code : o/n\n");
```

```

green();
scanf_s("%c", &validerModif, 1);
viderTampon();
reset();

if (validerModif == 'o')
{
    printf("L'ISBN a bien ete modifie\n");
    fseek(pTabLivres, -1 * (long)sizeof(newLivre),
SEEK_CUR);
    strcpy_s(newLivre.code, TAILLE_CODE,
newLivre.code);
    fwrite(&newLivre, sizeof(newLivre), 1,
pTabLivres);

}

else {
    system("cls");
    choixStock = AfficherEtDemandeChoixStock();
}
}
break;

case 2:
    printf("Entrer le titre du livre actuel: \n");
    red();
    gets_s(livreModifie.titre, TAILLE_TITRE);
    reset();

```

```

        while (fread_s(&newLivre, sizeof(newLivre), sizeof(newLivre), 1,
pTabLivres) != 0 && strcmp(newLivre.titre, livreModifie.titre) != 0) {}

        if (feof(pTabLivres)) {
            printf("Livre introuvable\n");
            printf("Entrer le titre du livre actuel: \n");
            red();
            gets_s(livreModifie.titre, TAILLE_TITRE);
            reset();
        }
        else {
            printf("Entrer le nouveau titre :\n");
            red();
            gets_s(newLivre.titre, TAILLE_TITRE);
            reset();

            printf("Voulez-vous modifier ce titre ? : o/n\n");
            green();
            scanf_s("%c", &validerModif, 1);
            viderTampon();
            reset();

            if (validerModif == 'o') {
                printf("Le livre a bien ete modifie\n");
                fseek(pTabLivres, -1 * (long)sizeof(newLivre),
SEEK_CUR);

                strcpy_s(newLivre.titre, TAILLE_TITRE,
newLivre.titre);

                fwrite(&newLivre, sizeof(newLivre), 1,
pTabLivres);
            }
        }
    }
}

```

```

        }

        else
        {
            system("cls");
            choixStock = AfficherEtDemandeChoixStock();
        }
    }
    break;

case 3:

    printf("Entrer l'auteur actuel: \n");
    red();
    gets_s(livreModifie.auteur, TAILLE_AUTEUR);
    reset();

    while (fread_s(&newLivre, sizeof(newLivre), sizeof(newLivre), 1,
pTabLivres) != 0 && strcmp(newLivre.auteur, livreModifie.auteur) != 0) {}

    if (feof(pTabLivres)) {
        printf("Livre introuvable\n");
        printf("Entrer l'auteur actuel: \n");
        red();
        gets_s(livreModifie.auteur, TAILLE_AUTEUR);
        reset();
    }
    else {
        printf("Entrer le nouvel auteur :\n");
        red();
        gets_s(newLivre.auteur, TAILLE_AUTEUR);
        reset();
    }

```

```

printf("Voulez-vous modifier l'auteur du livre ? : o/n\n");
green();
scanf_s("%c", &validerModif, 1);
viderTampon();
reset();

if (validerModif == 'o') {
    printf("L'auteur a bien ete modifie\n");
    fseek(pTabLivres, -1 * (long)sizeof(newLivre), SEEK_CUR);
    strcpy_s(newLivre.auteur, TAILLE_AUTEUR, newLivre.auteur);
    fwrite(&newLivre, sizeof(newLivre), 1, pTabLivres);

}

else
{
    system("cls");
    choixStock = AfficherEtDemandeChoixStock();
}
}
break;

```

case 4:

```

printf("Entrer l'année de parution actuelle: \n");
red();
scanf_s("%d", &livreModifie.date);
viderTampon();
reset();

```

```

while (fread_s(&newLivre, sizeof(newLivre), sizeof(newLivre), 1,

```

```
pTabLivres) != 0 && newLivre.date != livreModifie.date) {}
```

```
    if (feof(pTabLivres)) {
```

```
        printf("Livre introuvable\n");
```

```
        printf("Entrer l'annee de parution actuelle: \n");
```

```
        red();
```

```
        scanf_s("%d", &livreModifie.date);
```

```
        viderTampon();
```

```
        reset();
```

```
    }
```

```
    else {
```

```
        printf("Entrer la nouvelle annee de parution : \n");
```

```
        red();
```

```
        scanf_s("%d", &newLivre.date);
```

```
        viderTampon();
```

```
        reset();
```

```
        printf("Voulez-vous modifier ce titre ? : o/n\n");
```

```
        green();
```

```
        scanf_s("%c", &validerModif, 1);
```

```
        viderTampon();
```

```
        reset();
```

```
        if (validerModif == 'o') {
```

```
            printf("Le livre a bien ete modifie\n");
```

```
            fseek(pTabLivres, -1 * (long)sizeof(newLivre),
```

```
            SEEK_CUR);
```

```
            fwrite(&newLivre, sizeof(newLivre), 1,
```

```
            pTabLivres);
```

```
        }
```

```

else
{
    system("cls");
    choixStock = AfficherEtDemandeChoixStock();
}
}
break;

```

case 5:

```

printf("Entrer l'editeur actuel: \n");
red();
gets_s(livreModifie.editeur, TAILLE_EDITEUR);
reset();

```

```

while (fread_s(&newLivre, sizeof(newLivre), sizeof(newLivre), 1,
pTabLivres) != 0 && strcmp(newLivre.editeur, livreModifie.editeur) != 0) {}

```

```

if (feof(pTabLivres)) {
    printf("Livre introuvable\n");
    printf("Entrer l'editeur actuel: \n");
    red();
    gets_s(livreModifie.editeur, TAILLE_EDITEUR);
    reset();
}

```

```

else {
    printf("Entrer le nouvel editeur : \n");
    red();
    gets_s(newLivre.editeur, TAILLE_EDITEUR);
    reset();
}

```

```

        printf("Voulez-vous modifier cet editeur ? : o/n\n");
        green();
        scanf_s("%c", &validerModif, 1);
        viderTampon();
        reset();

        if (validerModif == 'o') {
            printf("L'editeur a bien ete modifie\n");
            fseek(pTabLivres, -1 * (long)sizeof(newLivre),
SEEK_CUR);
            strcpy_s(newLivre.editeur, TAILLE_EDITEUR,
newLivre.editeur);
            fwrite(&newLivre, sizeof(newLivre), 1,
pTabLivres);

        }

        else
        {
            system("cls");
            choixStock = AfficherEtDemandeChoixStock();
        }

    }
    break;
}

fclose(pTabLivres);

};

void AfficherLesLivresDuStock(void) {
    FILE* pTabLivres;
    FILE* pTabEmprunts;

```



```

Emprunt emprunts;
Livre AfficheLivre;
char etat[TAILLE_STATUT];

fopen_s(&pTabLivres, "ouvrages.dat", "rb+");
fopen_s(&pTabEmprunts, "emprunts.dat", "rb+");
while (fread_s(&AfficheLivre, sizeof(AfficheLivre), sizeof(AfficheLivre), 1,
pTabLivres) != 0) {

    if (strcmp(AfficheLivre.code, "****") != 0) {

        while (fread_s(&emprunts, sizeof(emprunts), sizeof(emprunts),
1, pTabEmprunts) != 0 && strcmp(emprunts.code, AfficheLivre.code) != 0) {}

        if (strcmp(emprunts.code, AfficheLivre.code) == 0) {

            strcpy_s(etat, TAILLE_STATUT, "Emprunte");

        }
        else {

            strcpy_s(etat, TAILLE_STATUT, "Libre");

        }

        printf("%10s|%11s|%40s|%30s|%4d|%20s\n", etat,
AfficheLivre.code, AfficheLivre.titre, AfficheLivre.auteur, AfficheLivre.date,
AfficheLivre.editeur);

```

```

    }

    fclose(pTabEmprunts);
    fopen_s(pTabEmprunts, "emprunts.dat", "rb+");

};

fclose(pTabLivres);
fclose(pTabEmprunts);

};

void RechercherLivreDansStock(void) {
    FILE* pTabLivres;
    Livre recherche;
    Livre newRecherche;
    fopen_s(&pTabLivres, "ouvrages.dat", "rb");

    printf("Quel livre souhaitez-vous recherche ? Entrer l'ISBN: \n");
    red();
    gets_s(recherche.code, TAILLE_CODE);
    reset();

    while (fread_s(&newRecherche, sizeof(newRecherche), sizeof(newRecherche), 1,
pTabLivres) != 0 && strcmp(newRecherche.code, recherche.code) != 0) {};

    if (feof(pTabLivres)) {
        printf("Livre introuvable\n");
    }
}

```

```

        // cas erreur while
    }

    else {

        cyan();

        printf("%20s|%10s|%4d|%20s\n", newRecherche.titre,
newRecherche.auteur, newRecherche.date, newRecherche.editeur);

        reset();

    }

    fclose(pTabLivres);
}

```

// Fonctions pour la gestion des emprunts

```

void EmprunterLivre(void) {

    FILE* pTabEmprunts;

    FILE* pTabMembres;

    FILE* pTabLivres;

    Membre membre;

    Membre membreBaseDonnees;

    Livre livre;

    Livre livreBaseDonnees;

    Emprunt livreEmprunte;

    Emprunt LivreBaseDonnees;

    int nbEmprunts = 1;

    float totalEmprunt = 0;

    int dateAjd = obtenirDateYYYYMMDDDuJour();

    int ecartJours;

    char validation;

    char payerAmende;

    bool payement = true;
}

```

```
bool membreExistant = false;
```

```
fopen_s(&pTabEmprunts, "emprunts.dat", "rb+");
```

```
fopen_s(&pTabMembres, "membres.dat", "rb+");
```

```
fopen_s(&pTabLivres, "ouvrages.dat", "rb+");
```

```
do {
```

```
    printf("Entrer le numero du membre : \n");
```

```
    red();
```

```
    scanf_s("%d", &membre.numMembre);
```

```
    viderTampon();
```

```
    reset();
```

```
    while(fread_s(&membreBaseDonnees, sizeof(membreBaseDonnees),  
sizeof(membreBaseDonnees), 1, pTabMembres) != 0 && membre.numMembre !=  
membreBaseDonnees.numMembre){}
```

```
    if (feof(pTabMembres)) {
```

```
        printf("Membre introuvable\n");
```

```
        printf("Voulez-vous creer un membre ? : (o/n)\n");
```

```
        green();
```

```
        scanf_s("%c", &validation, 1);
```

```
        reset();
```

```
        if (validation == 'o') {
```

```
            fclose(pTabMembres);
```

```
            AjouterMembre();
```

```
        }
```

```

        else {
            fclose(pTabMembres);
            membreExistant = true;
        }
    }
} while (membreExistant);

```

```

printf("Entrer l'ISBN du livre a emprunte :\n");
red();
gets_s(livreEmprunte.code, TAILLE_CODE);
reset();

while (fread_s(&LivreBaseDonnees, sizeof(LivreBaseDonnees),
sizeof(LivreBaseDonnees), 1, pTabEmprunts) != 0 && strcmp(LivreBaseDonnees.code,
livreEmprunte.code) != 0) {}

```

```

if (!feof(pTabEmprunts)) {
    printf("Le livre est deja emprunte\n");

}

```

```

while (fread_s(&LivreBaseDonnees, sizeof(LivreBaseDonnees),
sizeof(LivreBaseDonnees), 1, pTabEmprunts) != 0 && payement) {

```

```

    ecartJours = calculerNbJours(LivreBaseDonnees.dateEmprunt, dateAjd);

```

```

    if (livreEmprunte.numMembre == LivreBaseDonnees.numMembre) {

```

```

        nbEmprunts++;

        if (ecartJours > 14) {
            printf("Le membre a emprunte depuis trop
longtemps\n");

            printf("Une amende doit etre payee\n");
            printf("Voulez-vous la payee maintenant ? : o/n\n");
            green();
            scanf_s("%c", &payerAmende, 1);
            viderTampon();
            reset();

            if (payerAmende == 'o') {
                PayerAmende();
            }
            else {
                payement = false;
            }
        }

    }

}

if (livreEmprunte.numMembre > 10) {
    printf("Le membre a emprunte trop de livre");
}
else {
    fclose(pTabEmprunts);
    fopen_s(&pTabEmprunts, "emprunts.dat", "ab+");
}

```

```
printf("Voulez-vous emprunte ce livre ? : o/n\n");
green();
scanf_s("%c", &validation, 1);
viderTampon();
reset();
```

```
if (validation == 'o') {
    livreEmprunte.dateEmprunt = dateAjd;
    totalEmprunt = nbEmprunts * 0.75;
    printf("le livre a bien ete emprunte\n");
    printf("le montant de l'emprunt s'eleve a : %.2f@\n",
totalEmprunt);

    fwrite(&livreEmprunte, sizeof(livreEmprunte), 1,
pTabEmprunts);

}
else {

    system("cls");

}

}
```

```
fclose(pTabEmprunts);
fclose(pTabMembres);
fclose(pTabLivres);
```

```

};

void RemiseDesLivres(void) {
    FILE* pTabEmprunts;
    Emprunt remiseEmprunt;
    Emprunt empruntBaseDonnees;

    fopen_s(&pTabEmprunts, "emprunts.dat", "rb+");

    printf("Entrer le numero du membre qui veut rendre le livre:\n");
    red();
    scanf_s("%d", &remiseEmprunt.numMembre);
    reset();
    viderTampon();

    printf("Entrer le code ISBN du livre a rendre : \n");
    red();
    gets_s(remiseEmprunt.code, TAILLE_CODE);
    reset();

    while (fread_s(&empruntBaseDonnees, sizeof(empruntBaseDonnees),
sizeof(empruntBaseDonnees), 1, pTabEmprunts) != 0 && strcmp(remiseEmprunt.code,
empruntBaseDonnees.code) != 0) {}

    if (feof(pTabEmprunts)) {
        printf("Emprunt introuvable");
    }

    else {
        strcpy_s(remiseEmprunt.code, TAILLE_CODE, "****");
        fseek(pTabEmprunts, -1 * (long)sizeof(remiseEmprunt), SEEK_CUR);
        fwrite(&remiseEmprunt, sizeof(remiseEmprunt), 1, pTabEmprunts);
    }
}

```



```

        printf("Le livre est de nouveau disponible\n");
    }

    fclose(pTabEmprunts);

}

void PayerAmende(void) {
    FILE* pTabMembres;
    FILE* pTabEmprunts;
    Membre membre;
    Emprunt empruntLivre;
    Emprunt empruntLivre_bd;
    char validation_payer_amende;
    int ecart_date;
    int nb_calcul_amende;
    float amende;

    int date_auj = obtenirDateYYYYMMDDDuJour();
    bool decideDePayer = true;
    fopen_s(&pTabMembres, "membres.dat", "rb+");
    fopen_s(&pTabEmprunts, "emprunts.dat", "rb+");

    printf("Entrez le numero du membre qui veut payer une amende:\n");
    red();
    scanf_s("%d", &empruntLivre.numMembre);
    reset();
    viderTampon();
    do
    {

```

```

        while (fread_s(&membre, sizeof(membre), sizeof(membre), 1,
pTabMembres) != 0 && empruntLivre.numMembre != membre.numMembre)
        {
            };
            if (feof(pTabMembres))
            {
                printf("Numero du membre invalide\n");
                printf("Entrez le numero du membre qui veut payer une
amende:\n");

                scanf_s("%d", &empruntLivre.numMembre);
                viderTampon();

            }
            fclose(pTabMembres);
            fopen_s(&pTabMembres, "membres.dat", "rb+");
        } while (empruntLivre.numMembre != membre.numMembre);

        printf("Entrez l'ISBN du livre :\n");
        red();
        gets_s(empruntLivre.code, TAILLE_CODE);
        reset();
        while (fread_s(&empruntLivre_bd, sizeof(empruntLivre_bd), sizeof(empruntLivre_bd),
1, pTabEmprunts) != 0 && decideDePayer)
        {
            ecart_date = calculerNbJours(empruntLivre_bd.dateEmprunt, date_auj);
            if (strcmp(empruntLivre.code, empruntLivre_bd.code) == 0)
            {
                if (ecart_date > 14)
                {
                    nb_calcul_amende = ecart_date - 14;
                    amende = nb_calcul_amende * 0.10;

```

```

maintenant?:(o/n)\n");

printf("Amende a payer : %.2f\n", amende);
printf("voulez-vous payer l'amende

green();
scanf_s("%c", &validation_payer_amende, 1);
reset();
viderTampon();
if (validation_payer_amende == 'o')
{
    amende = 0;
    empruntLivre_bd.dateEmprunt =
date_auj;
    fseek(pTabEmprunts, -1 *
(long)sizeof(empruntLivre_bd), SEEK_CUR);
    fwrite(&empruntLivre_bd,
sizeof(empruntLivre_bd), 1, pTabEmprunts);
}
else
{
    decideDePayer = false;
}

}

else
{
    printf("Aucune amende a payer\n");
}

}

}

```

```

        fclose(pTabMembres);

        fclose(pTabEmprunts);

    }

    void RechercherUnLivre (void) {

        FILE* pTabEmprunts;

        Emprunt recherche;

        Emprunt newRecherche;

        fopen_s(&pTabEmprunts, "emprunts.dat", "rb");


        printf("Quel emprunt souhaitez - vous recherche ? Entrez l'ISBN: \n");

        red();

        scanf_s("%s", &newRecherche.code, TAILLE_CODE);

        viderTampon();

        reset();


        while (fread_s(&recherche, sizeof(recherche), sizeof(recherche), 1, pTabEmprunts) != 0
        && strcmp(recherche.code, newRecherche.code) != 0) {};


        if (feof(pTabEmprunts)) {

            printf("Livre introuvable\n");

        }


        else {

            cyan();

            printf("%20d|%20d|%20s\n", recherche.dateEmprunt,
recherche.numMembre, recherche.code);

            reset();

        }

        fclose(pTabEmprunts);

    }

```

```

void AfficherListeLivresEmpruntes(void) {
    FILE* pTabEmprunts;
    Emprunt AfficheEmprunts;

    fopen_s(&pTabEmprunts, "emprunts.dat", "rb");

    while (fread_s(&AfficheEmprunts, sizeof(AfficheEmprunts), sizeof(AfficheEmprunts),
1, pTabEmprunts) != 0) {

        if (strcmp(AfficheEmprunts.code, "****") == 0) {}

        else {

            printf("%20d|%20d|%20s\n", AfficheEmprunts.dateEmprunt,
AfficheEmprunts.numMembre, AfficheEmprunts.code);

        }

    };

    fclose(pTabEmprunts);

};

// Fonctions pour la gestion des membres
void IdentifierMembre(void) {
    FILE* pTabMembres;
    Membre membre;
    Membre newMembre;

    fopen_s(&pTabMembres, "membres.dat", "rb+");

```

```

printf("Entrer le numero du membre : \n");
red();
scanf_s("%d", &membre.numMembre);
viderTampon();
reset();

while (fread_s(&newMembre, sizeof(newMembre), sizeof(newMembre), 1,
pTabMembres) != 0 && newMembre.numMembre != membre.numMembre) {}

if (feof(pTabMembres)) {
    printf("Membre introuvable\n");
    // Ajouter membre
}
else {
    cyan();
    printf("%15s\t%15s\t%20s\n", newMembre.nom, newMembre.prenom,
newMembre.adresse);
    reset();
}
fclose(pTabMembres);
};

void AjouterMembre(void) {
    FILE* pTabMembres;
    Membre membre;
    Membre membreBD;
    char confirmation;
    bool nouveauMembre;

    do
    {

```

```

nouveauMembre = true;

fopen_s(&pTabMembres, "membres.dat", "ab + ");

printf("Entrez le numero du membre :\n");
red();
scanf_s("%d", &membre.numMembre);
reset();
viderTampon();

while (fread_s(&membreBD, sizeof(membreBD), sizeof(membreBD), 1,
pTabMembres) != 0)
{
    if (membre.numMembre == membreBD.numMembre)
    {
        printf("Le numero existe deja\n");
        nouveauMembre = false;
    }
}
fclose(pTabMembres);

} while (!nouveauMembre);

fopen_s(&pTabMembres, "membres.dat", "ab+");

printf("Entrer le nom du membre : \n");
red();
gets_s(membre.nom, TAILLE_NOM);
reset();

```

```
printf("Entrer le prenom du membre : \n");  
red();  
gets_s(membre.prenom, TAILLE_PRENOM);  
reset();
```

```
printf("Entrer l'adresse du membre : \n");  
red();  
gets_s(membre.adresse, TAILLE_ADRESSE);  
reset();
```

```
printf("Voulez-vous ajouter ce membre ? : (o/n)\n");  
green();  
scanf_s("%c", &confirmation, 1);  
reset();  
viderTampon();
```

```
if (confirmation == 'o') {  
    membre.dateAdhesion = obtenirDateYYYYMMDDDuJour();  
    fwrite(&membre, sizeof(membre), 1, pTabMembres);  
    printf("Le membre a bien ete ajoute :)\n");  
  
}  
else {  
    system("cls");  
  
}  
fclose(pTabMembres);
```

```
};
```

```
void SupprimerMembre(void) {  
    FILE* pTabMembres;
```



```
Membre membreSupprime;
```

```
Membre membre;
```

```
char confirmation;
```

```
fopen_s(&pTabMembres, "membres.dat", "rb+");
```

```
printf("Entrer le numero du membre a supprimer :\n");
```

```
red();
```

```
scanf_s("%d", &membreSupprime.numMembre);
```

```
reset();
```

```
viderTampon();
```

```
while (fread_s(&membre, sizeof(membre), sizeof(membre), 1, pTabMembres) != 0 &&  
membre.numMembre != membreSupprime.numMembre) {}
```

```
if (feof(pTabMembres)) {
```

```
    printf("Membre introuvable\n");
```

```
}
```

```
else {
```

```
    membre.numMembre = 999;
```

```
    fseek(pTabMembres, -1 * (long)sizeof(membre), SEEK_CUR);
```

```
    fwrite(&membre, sizeof(membre), 1, pTabMembres);
```

```
}
```

```
printf("Voulez-vous supprimer ce membre ? : o/n\n");
```

```
green();
```

```
scanf_s("%c", &confirmation, 1);
```

```
viderTampon();
```

```
reset();
```

```

        if (confirmation == 'o') {
            printf("Ce membre a bien ete supprime\n");
        }

        else {
            system("cls");
        }
        fclose(pTabMembres);
    };

void ModifierMembre(void) {
    FILE* pTabMembres;
    Membre membreModifie;
    Membre newMembre;
    char validerModif;
    int choixModification;

    IdentifierMembre();
    fopen_s(&pTabMembres, "membres.dat", "rb+");

    printf("I.Modifier le numero du membre\n ");
    printf("II. Modifier le nom du membre\n");
    printf("III.Modifier le prenom du membre\n");
    printf("IV.Modifier l'adresse du membre\n");

    printf("Entrer votre choix : \n");
    red();
    scanf_s("%d", &choixModification);
    viderTampon();
    reset();
}

```

```
system("cls");
```

```
switch (choixModification)
```

```
{
```

```
case 1:
```

```
    printf("Entrer le numero actuel du membre: \n");
```

```
    red();
```

```
    scanf_s("%d", &membreModifie.numMembre);
```

```
    viderTampon();
```

```
    reset();
```

```
        while (fread_s(&newMembre, sizeof(newMembre), sizeof(newMembre), 1,  
pTabMembres) != 0 && newMembre.numMembre != membreModifie.numMembre) {}
```

```
    if (feof(pTabMembres)) {
```

```
        printf("Numero introuvable\n");
```

```
        printf("Entrer un nouveau numero : \n");
```

```
    }
```

```
    else {
```

```
        printf("Entrer le nouveau numero :\n");
```

```
        red();
```

```
        scanf_s("%d", &newMembre.numMembre);
```

```
        viderTampon();
```

```
        reset();
```

```
        printf("Voulez-vous modifier ce numero ? : o/n\n");
```

```
        green();
```

```
        scanf_s("%c", &validerModif, 1);
```

```
        viderTampon();
```

```
        reset();
```

```

        if (validerModif == 'o') {
            printf("Le numero a bien ete modifie\n");
            fseek(pTabMembres, -1 *
(long)sizeof(newMembre), SEEK_CUR);
            fwrite(&newMembre, sizeof(newMembre), 1,
pTabMembres);

        }

        else
        {
            system("cls");
        }
    }
    break;

case 2:

    printf("Entrer le nom actuel du membre: \n");
    red();
    gets_s(membreModifie.nom, TAILLE_NOM);
    reset();

    while (fread_s(&newMembre, sizeof(newMembre), sizeof(newMembre), 1,
pTabMembres) != 0 && strcmp(newMembre.nom, membreModifie.nom) != 0) {}

    if (feof(pTabMembres)) {
        printf("Nom introuvable\n");
        printf("Entrer le nom actuel du membre: \n");
        red();
    }

```

```

        gets_s(membreModifie.nom, TAILLE_NOM);
        reset();
    }
    else {

        printf("Entrer le nouveau nom :\n");
        red();
        gets_s(newMembre.nom, TAILLE_NOM);
        reset();

        printf("Voulez-vous modifier ce nom ? : o/n\n");
        green();
        scanf_s("%c", &validerModif, 1);
        viderTampon();
        reset();

        if (validerModif == 'o') {
            printf("Le nom a bien ete modifie\n");
            fseek(pTabMembres, -1 *
(long)sizeof(newMembre), SEEK_CUR);
            strcpy_s(newMembre.nom, TAILLE_NOM,
newMembre.nom);
            fwrite(&newMembre, sizeof(newMembre), 1,
pTabMembres);

        }

        else
        {
            system("cls");
        }
    }

```

```

    }
    break;

case 3:

    printf("Entrer le prenom actuel du membre: \n");
    red();
    gets_s(membreModifie.prenom, TAILLE_PRENOM);
    reset();


    while (fread_s(&newMembre, sizeof(newMembre), sizeof(newMembre), 1,
pTabMembres) != 0 && strcmp(newMembre.prenom, membreModifie.prenom) != 0) {}


    if (feof(pTabMembres)) {
        printf("Prenom introuvable\n");
        printf("Entrer le prenom actuel du membre: \n");
        red();
        gets_s(membreModifie.prenom, TAILLE_PRENOM);
        reset();
    }
    else {

        printf("Entrer le nouveau prenom :\n");
        red();
        gets_s(newMembre.prenom, TAILLE_PRENOM);
        reset();


        printf("Voulez-vous modifier ce nom ? : o/n\n");
        green();
        scanf_s("%c", &validerModif, 1);
        viderTampon();
        reset();
    }
}

```

```

        if (validerModif == 'o') {
            printf("Le nom a bien ete modifie\n");
            fseek(pTabMembres, -1 *
(long)sizeof(newMembre), SEEK_CUR);
            strcpy_s(newMembre.prenom, TAILLE_PRENOM,
newMembre.prenom);
            fwrite(&newMembre, sizeof(newMembre), 1,
pTabMembres);

        }

        else
        {
            system("cls");
        }
    }
    break;

case 4:

    printf("Entrer l'adresse actuelle du membre: \n");
    red();
    gets_s(membreModifie.adresse, TAILLE_ADRESSE);
    reset();

    while (fread_s(&newMembre, sizeof(newMembre), sizeof(newMembre), 1,
pTabMembres) != 0 && strcmp(newMembre.adresse, membreModifie.adresse) != 0) {}

    if (feof(pTabMembres)) {
        printf("Adresse introuvable\n");
        printf("Entrer l'adresse actuelle du membre: \n");
        red();
        gets_s(membreModifie.adresse, TAILLE_ADRESSE);
    }

```

```

        reset();
    }
    else {

        printf("Entrer la nouvelle adresse :\n");
        red();
        gets_s(newMembre.adresse, TAILLE_ADRESSE);
        reset();

        printf("Voulez-vous modifier cette adresse ? : o/n\n");
        green();
        scanf_s("%c", &validerModif, 1);
        viderTampon();
        reset();

        if (validerModif == 'o') {
            printf("L'adresse a bien ete modifiee\n");
            fseek(pTabMembres, -1 *
(long)sizeof(newMembre), SEEK_CUR);
            strcpy_s(newMembre.adresse, TAILLE_ADRESSE,
newMembre.adresse);
            fwrite(&newMembre, sizeof(newMembre), 1,
pTabMembres);

        }

        else
        {
            system("cls");
        }
    }
    break;

```



```

    }
    fclose(pTabMembres);
};

void AfficherListeMembres(void) {
    FILE* pTabMembres;
    Membre AfficheMembre;

    fopen_s(&pTabMembres, "membres.dat", "rb+");

    while (fread_s(&AfficheMembre, sizeof(AfficheMembre), sizeof(AfficheMembre), 1,
pTabMembres) != 0) {

        if (AfficheMembre.numMembre == 999) {}

        else {

            printf("%d|%40s|%30s|%30s|\n",
AfficheMembre.numMembre, AfficheMembre.nom, AfficheMembre.prenom,
AfficheMembre.adresse, AfficheMembre.dateAdhesion);

        }

    };

    fclose(pTabMembres);
};

```

3.1. Structures et énumérations

Structure livres :

```

struct livre {
    char code[TAILLE_CODE];           = ISBN DU LIVRE
    char titre[TAILLE_TITRE];         = TITRE DU LIVRE
    char auteur[TAILLE_AUTEUR];       = AUTEUR DU LIVRE
    int date;                         = ANNÉE DE PARUTION
    char editeur[TAILLE_EDITEUR];     = ÉDITEUR DU LIVRE
};

```

Structure emprunts :

```
struct emprunt {  
    int dateEmprunt;           = DATE D'EMPRUNT  
    int numMembre;             = NUMÉRO DE L'EMPRUNTEUR  
    char code[TAILLE_CODE];    = ISBN DU LIVRE EMPRUNTÉ  
};
```

Structure membres :

```
struct membre {  
    int numMembre;             = NUMÉRO DU MEMBRE  
    char nom[TAILLE_NOM];      = NOM DU MEMBRE  
    char prenom[TAILLE_PRENOM]; = PRÉNOM DU MEMBRE  
    char adresse[TAILLE_ADRESSE]; = ADRESSE DU MEMBRE  
    int dateAdhesion;          = DATE D'ADHÉSION DU MEMBRE  
};
```

3.2. Prototypes

```
bool FctOuvertureFichiers(void);
```

Entrée : rien

Sortie : Un booléen

Effets : La fonction ouvre les fichiers si tout va bien sinon elle n'ouvre rien

```
int AfficheMenuEtDemandeChoix(void);
```

Entrée : rien

Sortie : Un entier (le numéro du choix)

Effets : La fonction affiche le premier menu et demande le premier choix

// Fonctions pour le stock

```
int AfficherEtDemandeChoixStock(void);
```

Entrée : rien

Sortie : Un entier (le numéro du choix)

Effets : La fonction affiche le menu du stock et demande le choix

```
void GestionDuStock(void);
```

Entrée : rien

Sortie : rien

Effets : La fonction affiche gère le menu du stock grâce à un switch

```
void ajouterLivreAuStock(void);
```

Entrée : rien

Sortie : rien

Effets : La fonction demande toutes les informations d'un livre et les ajoutent au fichier

`void SupprimerLivreDuStock(void);`

Entrée : rien

Sortie : rien

Effets : La fonction demande l'ISBN du livre et supprime la ligne correspondante dans le fichier

`void ModifierLivreStock(void);`

Entrée : rien

Sortie : rien

Effets : La fonction demande les modifications et les effectuent dans le fichier

`void AfficherLesLivresDuStock(void);`

Entrée : rien

Sortie : rien

Effets : La fonction affiche le contenu du fichier des livres

`void RechercherLivreDansStock(void);`

Entrée : rien

Sortie : rien

Effets : La fonction demande l'ISBN d'un livre et affiche les données correspondantes

// Fonctions pour les emprunts

`int AfficherEtDemandeChoixEmprunts(void);`

Entrée : rien

Sortie : Un entier (le numéro du choix)

Effets : La fonction affiche le menu des emprunts et demande le choix

`void GestionDesEmprunts(void);`

Entrée : rien

Sortie : rien

Effets : La fonction gère le menu des emprunts grâce à un switch

`void EmprunterLivre(void);`

Entrée : rien

Sortie : rien

Effets : La fonction fait ce qu'il faut pour emprunter un livre

`void RemiseDesLivres(void);`

Entrée : rien

Sortie : rien

Effets : La fonction demande le numéro du membre et l'ISBN du livre à rendre puis rend celui-ci

```
void PayerAmende(void);
```

Entrée : rien

Sortie : rien

Effets : La fonction demande le numéro du membre et regarde si il a une amende à payer

```
void RechercherUnLivre(void);
```

Entrée : rien

Sortie : rien

Effets : La fonction demande l'ISBN d'un livre emprunté et affiche les données correspondantes

```
void AfficherListeLivresEmpruntes(void);
```

Entrée : rien

Sortie : rien

Effets : La fonction affiche le contenu du fichier emprunts

// Fonctions pour les membres

```
int AfficherEtDemandeChoixMembres(void);
```

Entrée : rien

Sortie : Un entier (le numéro du choix)

Effets : La fonction affiche le menu des membres et demande le choix

```
void GestionDesMembres(void);
```

Entrée : rien

Sortie : rien

Effets : La fonction gère le menu membre grâce à un switch

```
void IdentifierMembre(void);
```

Entrée : rien

Sortie : rien

Effets : La fonction demande le numéro du membre et affiche ses données

```
void AjouterMembre(void);
```

Entrée : rien

Sortie : rien

Effets : La fonction demande les informations nécessaire et ajoute le membre au fichier

```
void SupprimerMembre(void);
```

Entrée : rien

Sortie : rien

Effets : La fonction demande le numéro du membre et celle-ci supprime ce membre du fichier

`void ModifierMembre(void);`

Entrée : rien

Sortie : rien

Effets : La fonction demande le numéro du membre et modifie les informations souhaitées

`void AfficherListeMembres(void);`

Entrée : rien

Sortie : rien

Effets : La fonction affiche le contenu du fichier membre

`int obtenir_annee(void);`

Entrée : rien

Sortie : rien

Effets : La fonction permet de déterminer si l'année rentrée existe bien

`void viderTampon(void);`

Entrée : rien

Sortie : rien

Effets : La fonction permet de vider le buffer

`void extraireDepuisDateYYYYMMDD(int date, int* year, int* month, int* day);`

Entrée : Un entier (la date), 3 pointeurs d'entiers (année, mois, jour)

Sortie : rien

Effets : La fonction permet de décortiquer une date stockée sous la forme d'un entier pour pouvoir avoir l'année, le mois et le jour séparé

`int calculerNbJours(int dateStartYYYYMMDD, int dateEndYYYYMMDD);`

Entrée : 2 entiers (date de départ, date de fin)

Sortie : Un entier (l'écart entre les 2 dates)

Effets : La fonction permet calculer l'écart entre 2 dates

`int obtenirDateYYYYMMDDDuJour(void);`

Entrée : rien

Sortie : Un entier (la date du jour stockée sous la forme « AAAA-MM-JJ »)

Effets : La fonction permet d'obtenir la date du jour

4. Rétrospective

Les forces de mon programme sont :

- Le programme utilise des couleurs lorsque l'on fait un choix.
- Le programme est clair, tout le monde peut l'utiliser.
- Le code est clair à lire et à comprendre.
- Toutes les fonctions remplissent leur job.

Les faiblesses de mon programme sont :

- Le programme peut – être plus robuste.
- Le programme peut contenir plus de fonctionnalités.
- Le programme manque de cas d'erreurs.

Toutes les fonctionnalités ont pu être implémentées mais il manque juste des cas d'erreurs dans certaines fonctions.

5. Perspectives

- On peut déjà commencer par compléter avec les cas d'erreurs qu'il manque.
- On pourrait créer un fichiers qui enregistre les emprunts.
- On pourrait créer une fonctionnalité qui indique quels livres ont été le plus de fois empruntés.
- On pourrait créer une fonctionnalité où l'on peut modifier la date de remise d'un livre pendant que celui-ci est emprunté.

6. Annexe : rencontre avec les différents acteurs

« Nous sommes une petite communauté bénévoles qui met à disposition des livres à un prix attractif. Nous avons regroupé un ensemble d'ouvrages venant de quelques déstockages. Nous les avons tous listés avec leur référence (il s'agit d'un code ISBN à 10 chiffres), l'auteur, l'éditeur, l'année de parution, le titre. La bibliothèque ne possède pas beaucoup de place, ainsi nous avons comme parti pris que nous ne posséderons jamais deux fois le même livre.

Avant que la bibliothèque ne soit ouverte, la première chose que nous souhaiterions c'est de mettre en place un système qui nous permette de lister tous ces ouvrages, de pouvoir en ajouter de nouveaux, de pouvoir modifier le contenu d'un ouvrage ou de pouvoir en supprimer. »

Peter : Président

« Nous souhaiterions également pouvoir indiquer leur statut. S'ils sont libres où empruntés. Bien entendu, chaque personne qui emprunterait un livre serait obligatoirement enregistrée dans le système. Ils seraient membres en quelque sorte. Nous conserverions leur nom, prénom, adresse, ainsi qu'un numéro d'identification et leur date d'adhésion à la bibliothèque. L'enregistrement est gratuit. . Tout comme les livres nous pourrions modifier les données d'un membre, ou le supprimer ou en ajouter un nouveau. »

Auguste : gestionnaire des stocks

« Les emprunts fonctionnent de la manière suivante. Pour chaque livre, l'emprunt coûte 0,75€. L'emprunt est fixé à deux semaines. Un membre ne peut emprunter plus de 10 livres à la fois. Une amende de 0,10€ par jour est notée pour chaque livre pour chaque semaine de retard. Si un membre possède une amende impayée et/ou un livre en retard, il ne peut plus emprunter de nouveau livre.

J'imagine un écran pour l'emprunt où l'on insère le numéro du membre et les différents livres qu'il emprunte. S'affiche alors le prix à payer. Nous validerions nous même le paiement puisque nous n'accepterons que l'argent liquide.

Il serait intéressant d'avoir une partie dans le programme destinée à la remise des livres. On insère leur ISBN et on sait qu'ils sont à nouveau disponibles. S'il y a une amende à payer, elle est indiquée à l'écran. Nous validerions ensuite si l'utilisateur a bien payé son amende. »

Ce qui est également très sollicité, c'est la recherche. Ne fusse que sur un code ISBN. Pour que l'on puisse déterminer si nous possédons l'ouvrage, s'il est disponible ou sinon à quelle date il est supposé revenir. »

Rosie : accueil

« La priorité du logiciel doit être la gestion du stock. Puisque nous gardons une comptabilité papier, la gestion des prix et des amendes peut être intégrée dans un second temps. »

Peter : Président

«Je pense pouvoir vous fournir le fichier complet avec le contenu complet des références. Pensez-vous qu'il soit possible que votre logiciel mette à jour ce fichier texte ? Ainsi nous pourrions également toujours pouvoir ressortir la liste des ouvrages que nous possédons. Il faudrait en fait que le logiciel conserve également un fichier pour les membres. Et je pense très probablement pour les emprunts en cours. »

David: Statistiques